A CIFAR Residual Network Under Five Million Parameters

Pengxiao Yu (py2352) | Individual project
Deep Learning, Fall 2026
Kaggle team: Pengxiao | Username: pengxiao0214

Public code and executed notebook:
https://github.com/Pengxiao-Yu/dl-project1-cifar-resnet

1. Overview

This project adapts a basic-block residual network to 32 x 32 color images under a strict model-size budget. It uses a narrow CIFAR stem, four residual stages and global average pooling. All weights are initialized from scratch; the training pipeline uses only images and labels provided in the competition archive.

The network has **4,368,690 trainable parameters**. In one fixed-seed run of 80 epochs, the selected checkpoint at epoch 77 achieved **95.10% accuracy** on the fixed validation partition. Kaggle late evaluation returned private accuracy 81.70% and public accuracy 83.40%. This was submitted after the deadline and is not an on-time ranked entry.

Measured quantity	Result
Trainable parameters	4,368,690 / 5,000,000
Training / validation examples	45,000 / 5,000
Selected validation accuracy	95.10% (4,755 / 5,000)
Kaggle private score (late)	81.70%
Kaggle public score (late)	83.40%
Epochs / selected epoch	80 / 77
Epoch training + validation time	60.64 min on mps (Apple M1 Pro)

The budget check, residual-path identity, finite gradients, checkpoint round-trip and augmentation isolation were verified before the experiment. Formal results come from the full data run, while synthetic engineering checks are recorded separately.

AI assistance disclosure: OpenAI Codex assisted implementation, experiment orchestration and report drafting. All numerical results in this report derive from saved executed artifacts.

2. Model and Data Protocol

Residual architecture

Each basic block computes ReLU(S(x) + BN(Conv3x3(ReLU(BN(Conv3x3(x)))))). The skip path is an identity unless spatial size or channels change, when a stride-matched 1 x 1 convolution and batch normalization project the input. This follows the residual-learning formulation of He et al. [1].

Stage	Output shape	Blocks / operation
Input	3 x 32 x 32	RGB competition image
Stem	40 x 32 x 32	3 x 3 conv, stride 1; BN; ReLU
Residual stage 1	40 x 32 x 32	2 basic blocks
Residual stage 2	80 x 16 x 16	2 blocks; first stride 2
Residual stage 3	160 x 8 x 8	2 blocks; first stride 2
Residual stage 4	320 x 4 x 4	2 blocks; first stride 2
Readout	10 logits	Global average pooling; linear layer

Design tradeoffs

The stride-one stem and absence of an early max pool preserve information at CIFAR resolution. Reducing base width to 40 controls the roughly quadratic cost of channel width while retaining four stages and eight residual blocks. Global average pooling avoids a large flattened classifier. Projection skips add parameters but provide a learned match at resolution transitions. This run evaluates the resulting design; it does not establish that any one choice is independently optimal.

Dataset and separation

The instructor archive contains 50,000 labeled training images and 2,000 unlabeled competition test images. Images are read in CSV order. A seed-2026 stratified 10% holdout creates disjoint training and validation index arrays. Channel means and standard deviations are computed using training images only. Validation receives normalization only and never updates optimizer or batch-normalization statistics. Hidden labels are not accessed.

Archive CRC checks, exact image shape, label range, unique IDs and test/sample ID order are checked before caching. The archive and split hashes are preserved in the data manifest. This competition dataset is a version of CIFAR-10; it must not be conflated with a separately evaluated standard CIFAR-10 test set [2].

3. Training and Model Selection

The model, optimizer and loss are implemented in PyTorch [4].

Setting	Value
Optimizer	SGD; momentum 0.9; Nesterov; weight decay 0.0005
Schedule	80 epochs; batch 128; initial LR 0.1; 5-epoch warm-up; cosine decay to 0.0001
Augmentation	Reflect padding 4; independent random 32 x 32 crop; horizontal flip probability 0.5
Objective	Cross entropy with label smoothing 0.05
Selection	Highest fixed-validation accuracy; ties use lower validation cross entropy
Initialization	Kaiming-normal convolutions; no pretrained weights; seed 2026

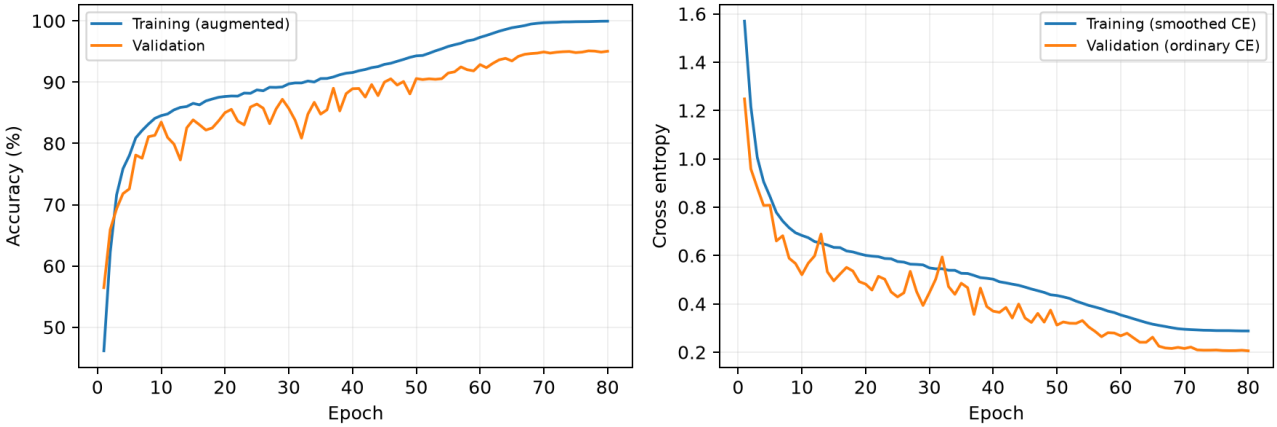


Figure 1. The actual epoch histories. Training accuracy includes random augmentation, and its loss includes label smoothing. Validation uses ordinary cross entropy on unaugmented images, so the two losses are not identical objectives.

Validation accuracy changed from 56.46% in the first epoch to a selected value of 95.10% at epoch 77. The final epoch accuracy was 95.04%. Checkpoint selection follows the predeclared validation rule rather than selecting the last epoch by default. The model used for inference is verified by a SHA256 hash.

Optimization and recoverability

Timing covers epoch training, validation and device synchronization; checkpoint I/O is excluded. The measured hardware is an Apple M1 Pro with 16 GiB of unified memory.

Each completed epoch writes the configuration, training/validation metrics, optimizer state and CPU/device RNG state. Checkpoints are written to a temporary file and replaced atomically. Resume checks the data hash, architecture, schedule, seed and batch size. This preserves a recoverable run, although cross-device kernels and hardware can still change floating-point results.

4. Validation Diagnostics

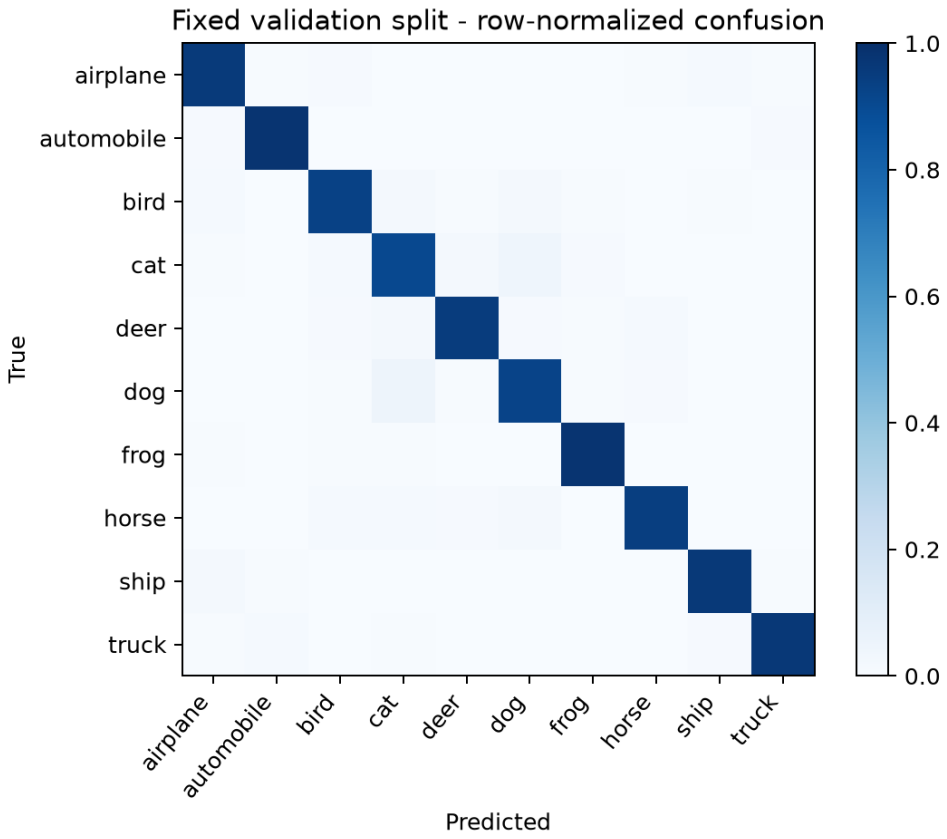


Figure 2. Confusion matrix on the fixed validation partition, normalized within each true class. All rows are computed from reloaded-checkpoint predictions.

Per-class accuracy

Class	Accuracy	Class	Accuracy
airplane	96.00%	dog	92.20%
automobile	98.20%	frog	98.20%
bird	93.20%	horse	94.40%
cat	90.60%	ship	96.40%
deer	95.00%	truck	96.80%

The lowest validation class accuracies were cat (90.60%), dog (92.20%), bird (93.20%). The strongest three classes were frog (98.20%), automobile (98.20%), truck (96.80%). These are observations of this split and run, not claims about every possible CIFAR-10 sample.

A descriptive Wilson interval for the observed proportion is [94.47%, 95.66%]. Because the checkpoint was selected on the same validation set, this is not an independent guarantee of generalization and does not quantify all model-selection or seed uncertainty.

5. Error Review and Limits

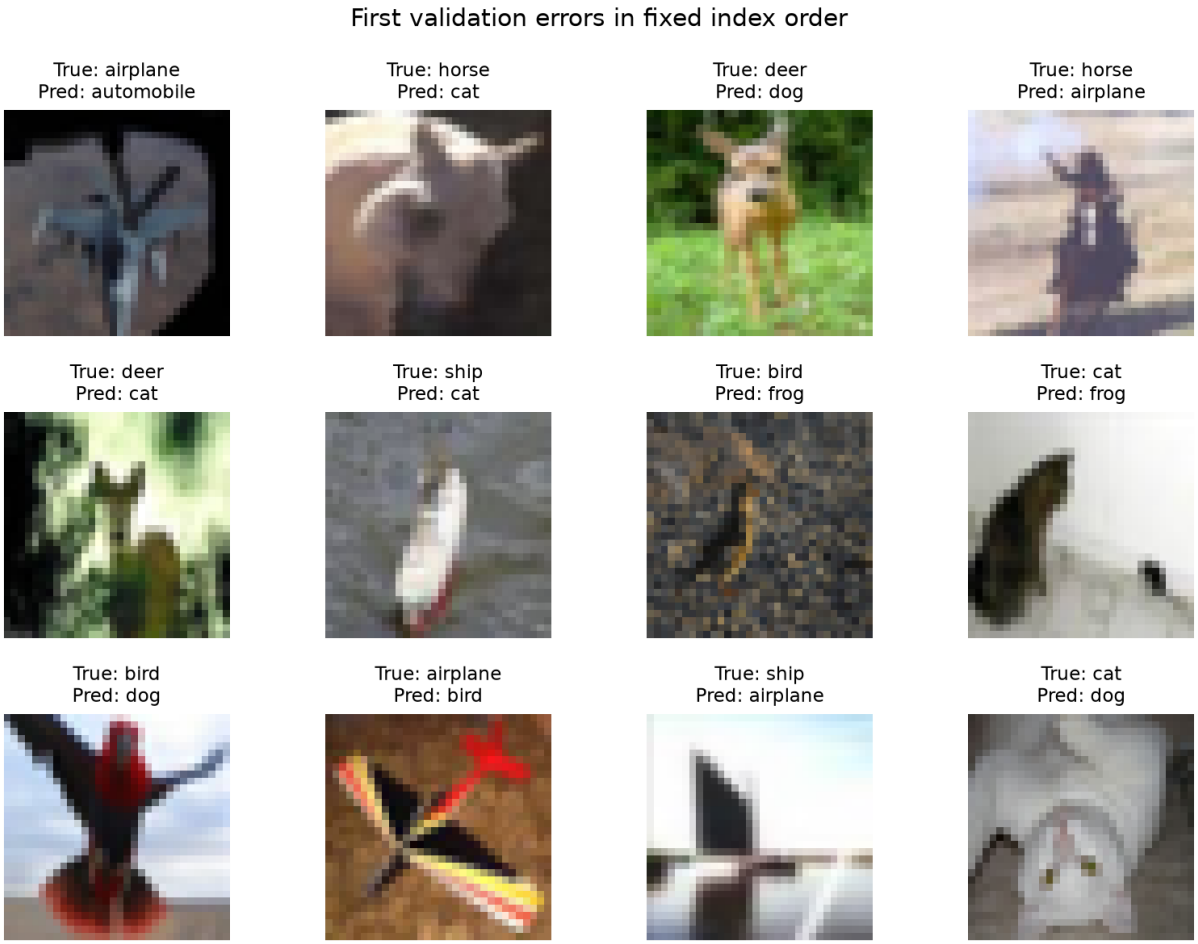


Figure 3. First 12 errors in ascending fixed validation-index order, shown with true and predicted class labels. Examples are selected mechanically rather than chosen to tell a favorable story.

What this experiment supports

The saved run verifies that a residual model can be trained from scratch within the parameter budget, and that the selected checkpoint reproduces the reported validation predictions. The learning history and errors provide evidence about this one architecture and split. They do not isolate the causal contribution of width, residual connections, augmentation or smoothing, because no matched ablation study was performed.

Limitations and next experiments

Only one training seed and one validation split are used. Validation is also used for checkpoint selection, which can make the selected number optimistic. No ensemble or pretraining is used. A useful extension would compare narrower widths and augmentation choices with equal compute and repeat the selected configuration across seeds. The final hidden-test score, when available, is the competition evaluation rather than an interchangeable validation statistic.

6. Reproducibility and References

Reproduction steps

1. Obtain the competition data.zip from Kaggle or the instructor attachment.
2. Install requirements and run code/checks.py.
3. Run code/prepare_data.py /path/to/data.zip --out data.
4. Run code/train.py --out results/main --epochs 80.
5. Run code/predict.py and code/analyze.py.
6. Execute Project1.ipynb from the repository root, inspect outputs, and upload the generated CSV to Kaggle.

Use --resume with the same settings after interruption. The prepared code package includes code, configuration, histories, summaries, figures and an executed notebook. Data archives and large checkpoints are excluded from Git; the model can be reproduced by training from scratch. The checkpoint and prediction hashes below identify this measured run. Training must occur on an allocated GPU session if using NYU HPC.

Evidence	Value
PyTorch	2.14.1
Platform	macOS-26.6-arm64-arm-64bit
Data ZIP SHA256	397e06e5752bfe0576d1258765bbe1d020cca02db2fc21442cbab6d50257ca51
Checkpoint SHA256	8a9b66c1b3a49e899b9a05b8b5ed9f78898f90fda708b8fc0099816ca08d66a1
Submission CSV SHA256	99372484f170abb603a332d753ab8d62e58c726bb745389cd71a05caa94d6413

Kaggle verification: https://www.kaggle.com/competitions/cifar-10-res-net-architecture-challenge-dl/submissions
Observed private score: 0.817; public score: 0.83400. Late evaluation; no on-time rank claimed. Team registration confirmed.
Submission: submission.csv (late submission).

References

[1] K. He, X. Zhang, S. Ren and J. Sun. Deep Residual Learning for Image Recognition. CVPR, 2016. https://arxiv.org/abs/1512.03385
[2] A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, 2009. https://www.cs.toronto.edu/~kriz/cifar.html
[3] K. Liu. pytorch-cifar. Instructor-linked background reference, consulted for context. https://github.com/kuangliu/pytorch-cifar
[4] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS, 2019. https://arxiv.org/abs/1912.01703

Implementation and attribution

The basic-block design follows [1]. Model, data validation, training, prediction and diagnostics code were authored for this project with OpenAI Codex assistance. No downloaded model weights or external image datasets were used in the formal experiment. The course PDF defines the project constraint and report requirements.